PROGRAMMING WITH DATA

TITLE- LIBRARY ASSISTANT

The chatbot I developed in this project is a library Assistant Chatbot. Its main purpose is to help users by responding to queries related to books, authors, genres and availability. The chatbot can have a conversational interface where users can ask for book recommendations, search for books by author or genre and receive friendly responses.

The chatbot operates in educational and information retrieval domain. It's designed to be simple, interactive and user friendly. It also supports conversational memory, allowing it to remember some personal details such as the user's name or favourite book and use this information later to create more fun interaction.

For Part 1, I followed the Coursera video, which showed how a chatbot uses a main loop to keep taking user input and only stops when the user types "exit" or "quit". I used enumerate () instead of list.index() because it makes it easier to loop through the list while also keeping track of the position of each item. Then I got feedback from my friends saying my chatbot was too general and didn't have a clear domain. Because of that later I came up with library assistant.

```
def chatbot():
    print("Welcome! Press bye to exit")

    while True:
        user_input = input("You: ").lower()

        if user_input == "bye":
            print("Chatbot: thank you for the interaction")
            break

        response_found = False

        for i, pattern in enumerate(precepts):
            if pattern in user_input:
                print("Chatbot:", responses[i])
                response_found = True
                break

        if not response_found:
            print("I'm sorry, could you rephrase?")

# Run the chatbot
chatbot()
```

Part 2 took a really long time, but I followed the instructions given in the template. I used dictionaries for intents, patterns, responses. For pattern recognition, I used regex-based matching and generate response. For the advanced regular expressions, my chatbot can remember names, and instead of using favourite colours, I changed it to favourite books. I used the pattern (?:\s+[A-Z][a-z]+){0,2} so that the chatbot can recognise names with two or three words. I got feedback saying my code was messy and needed better organisation, so I refactored the code into functions to separate pattern detection, response generation and input handling.

```
# Dynamic patterns for name and favorite book
dynamic_patterns = {
    r"(?:my name is|i am|call me)\s+([A-Z][a-z]+(?:\s+[A-Z][a-z]+){0,2})": "Nice to meet you, {name}!",
    r"(?:my favorite book is|my favourite book is|i like|book is|favourite is)\s+([A-Za-z]+(?:\s+[A-Za-z]+){0,2})": "{book}
}
```

For Parts 3 and 4, I stored all the intents, patterns, and responses in a JSON file. Based on the feedback I received, my chatbot relied too much on exact keywords, so to improve this, I used NLTK libraries. I applied tokenisation to split the user input into individual words. Then, I used stemming with PorterStemmer, which helps the chatbot match different word forms like “books” and “book”. I also implemented lemmatisation using WordNetLemmatizer for semantic matching. By combining both stemming and lemmatisation, the chatbot is better able to handle like more diverse users.

Test cases that show the chatbot behaviour:

Test case1: can help the user find books with help of author, title and genre

```
Welcome! Press bye to exit
You: hello
Chatbot: Hello! How can I assist you today?
You: im searching for a book
Chatbot: Sure, what book are you looking for? You can search by author, title or genre.
You: hmm can i search by author
Chatbot: Please enter the author's name.
You: no can i search by title
Chatbot: Please enter the title.
You: i want crime and punishment
Chatbot: Crime and Punishment is available to borrow.
You: can i search by genre too
Chatbot: Please enter the genre.
You: i was thinking horror but idk
Chatbot: Horror: The Little Cat is available to borrow.
You: ohh okay thank you
Chatbot: You are welcome!
You: bye
Chatbot: thank you for the interaction
```

The chatbot greets the user at the start and asks what type of search they want to dofor the book, such as by author, title, or genre. It then prompts the user to enter a book title and confirms the availability of *Crime and Punishment*. If the user chooses to search by genre, the chatbot asks for the genre and returns a matching horror book. It also responds politely when the user says thank you and exits the conversation when the user says “bye”.

Test case2: the chatbot can remember the user ‘s name and their favourite book.

```
Bot: Hello! I'm here to chat with you. Tell me your name or favorite book!
You: hello
Bot: Hello! How can I assist you today?
You: My name is James park
Bot: Nice to meet you, James park!
You: i want a book
Bot: Sure, what book are you looking for? You can search by author, title or genre.
You: hmmmm i want horror
Bot: Horror: The Little Cat is available to borrow.
You: i like No longer human
Bot: No longer human is a wonderful book!
You: is there dostoevsky
Bot: Crime and Punishment is available to borrow.
You: ok thank you
Bot: You are welcome!
You: bye
Bot: Goodbye! Have a great day.
```

The chatbot greets the user at the start, detects the user's name using a pattern, and stores it for later use. It then replies with a personalised message such as "Nice to meet you, James Park!". It also detects the user's favourite book using a pattern and returns "No longer human is a wonderful book!". The chatbot provides book recommendations or checks availability based on the book title or author. Finally, it ends the conversation when the user says "bye".

Test case 3: It limits the user to only 3 books and doesn't let the user borrow after that

```
==================================================
Library Chatbot - Borrow Limit System
Max books: 3
==================================================
You: hello
Bot: I'm doing good! How can i help you today?

You: i want to borrow some books
Bot: Okay! Tell me the author, title, or genre you want.

You: i want horror
Bot: In horror, The Little Cat is available to borrow.

You: borrow the little cat
Bot: Done! You borrowed 'the little cat'. You can borrow 2 more.

You: hmmm is there dostoevsky
Bot: dostoevsky:Crime and Punishment is available to borrow.

You: borrow crime and punishment
Bot: Done! You borrowed 'crime and punishment'. You can borrow 1 more.

You: i want to borrow another book
Bot: Sure, what book are you looking for? You can search by author, title or genre.

You: borrow pride and prejudice
Bot: Done! You borrowed 'pride and prejudice'. You can borrow 0 more.

You: borrow No longer human
Bot: Sorry! You already borrowed 3 books. Return one first!

You: ok thank you
Bot: Happy to help!

You: bye
Bot: Bye! Come back soon.
```

The chatbot confirms each book borrow and updates the remaining borrow count. Once the user has borrowed three books, it prevents any further borrowing and informs the user that they must return a book first. Finally, the chatbot exits politely when the user says "bye".

My advanced feature:

I implemented a borrowing system that can keep track of the books borrowed by the user during a session. The chatbot limits borrowing to three books, prevents the same book from borrowing twice and updates the remaining borrow count. Users are also

able to return books and the chatbot checks whether the book was actually borrowed before removing it. This prevents in logical errors and makes the system feel more realistic by handling invalid return attempts. Also, instead of reacting to single keywords the chatbot recognises the full commands. When an intent s matched, the chatbot selects a random response from list to avoid repetitive and makes the conversation flow natural.

The chatbot uses modular programming with many user-defined functions a constant configuration value to control the system limits. I applied iteration using a continuous while loop and nested for loop, combined with conditional logic (if/else) for control and command routing. String manipulation techniques such as case normalization, slicing, trimming, substring matching, joining and formatted strings are used to process and generate output. It can randomise using the random library and the program termination is handled using a break condition.

